Fonctions en Swift - Partie 2

420-249-AH - Programmation mobile iOS

1. Retourner plusieurs valeurs

Une fonction Swift peut retourner plusieurs informations à la fois grâce à un tuple.

Exemple :

```swift
func obtenirEtatBatterie(niveau: Int) -> (pourcentage: Int, etat: String) {
    if niveau <= 20 {
        return (niveau, "Critique")
    } else if niveau <= 50 {
        return (niveau, "Faible")
    } else if niveau <= 80 {
        return (niveau, "Normal")
    } else {
        return (niveau, "Excellent")
    }
}
```

Utilisation :

```swift
let resultat = obtenirEtatBatterie(niveau: 75)
print(resultat.pourcentage)
print(resultat.etat)
```

Résultat :

```
Console
75
Normal
```

2. Pourquoi retourner plusieurs valeurs ?

Dans une application mobile, un traitement peut produire plusieurs résultats.

Traitement	Résultats
`Position GPS`	latitude + longitude
`Authentification`	succès + message
`Téléchargement`	progression + état
`Validation`	résultat + message d'erreur

Au lieu de créer plusieurs fonctions, un tuple peut parfois regrouper ces informations.

3. Tuple simple

```
func obtenirPosition() -> (Double, Double) {
    return (45.5019, -73.5674)
}
```

Utilisation :

```
let position = obtenirPosition()
print(position.0)
print(position.1)
```

Résultat :

```
Console
45.5019
-73.5674
```

Cette syntaxe fonctionne, mais elle n'est pas très descriptive.

4. Tuple nommé

Il est préférable de donner un nom aux valeurs retournées.

```
func obtenirPosition() -> (
    latitude: Double,
    longitude: Double
) {
    return (45.5019, -73.5674)
}
```

Utilisation :

```
let position = obtenirPosition()
print(position.latitude)
print(position.longitude)
```

Le code devient beaucoup plus lisible.

5. Décomposer un tuple

On peut également récupérer directement les différentes valeurs.

```
let (latitude, longitude) = obtenirPosition()
print(latitude)
print(longitude)
```

On appelle cela décomposer un tuple.

6. Ignorer une valeur retournée

Supposons :

```
func obtenirProfil() -> (
    nom: String,
    age: Int,
    ville: String
) {
    return ("Alex", 25, "Montréal")
}
```

Si l'âge ne nous intéresse pas :

```
let (nom, _, ville) = obtenirProfil()
```

Le symbole _ permet d'ignorer une valeur.

7. Exemple — Validation d'un formulaire

Une fonction peut retourner : valide + message.

Exemple :

```
func validerCourriel(_ courriel: String) -> (
    valide: Bool,
    message: String
) {
    if courriel.isEmpty {
        return (false, "Le courriel est vide")
    }
    if !courriel.contains("@") {
        return (false, "Le courriel est invalide")
    }
    return (true, "Courriel valide")
}
```

Utilisation :

```
let resultat = validerCourriel("alex@email.com")
print(resultat.valide)
print(resultat.message)
```

8. Les fonctions ont aussi un type

En Swift, une fonction possède un type. Prenons :

```
func additionner(a: Int, b: Int) -> Int {
    return a + b
}
```

Le type de cette fonction est :

```
(Int, Int) -> Int
```

On peut lire :

```
reçoit deux Int
        ↓
retourne un Int
```

9. Autres exemples de types de fonctions

```
func afficherMessage(message: String) {
    print(message)
}
```

Type :

```
(String) -> Void
```

Cela signifie : reçoit un String, ne retourne aucune valeur utile.

```
func estValide(code: Int) -> Bool {
    return code == 1234
}
```

Type :

```
(Int) -> Bool
```

10. Le type Void

Une fonction qui ne retourne aucune valeur explicite retourne conceptuellement :

```
Void
```

Exemple :

```
func afficherBonjour() {
    print("Bonjour")
}
```

Son type peut être représenté comme :

```
() -> Void
```

11. Une fonction peut être stockée dans une variable

En Swift, une fonction peut être traitée comme une valeur.

Exemple :

```swift
func additionner(a: Int, b: Int) -> Int {
    return a + b
}
```

On peut écrire :

```swift
let operation = additionner
```

Puis :

```swift
let resultat = operation(10, 5)
print(resultat)
```

Résultat :

```
Console
15
```

12. Que contient operation ?

Avec :

```swift
let operation = additionner
```

operation ne contient pas le résultat de la fonction. Elle contient une référence vers la fonction elle-même.

On peut donc faire :

```swift
operation(10, 5)
```

comme on aurait fait :

```swift
additionner(a: 10, b: 5)
```

13. Déclarer explicitement le type d'une fonction

On peut écrire :

```swift
let operation: (Int, Int) -> Int = additionner
```

Cela indique clairement :

```
operation
    ↓
reçoit deux Int
    ↓
retourne un Int
```

14. Exemple avec plusieurs opérations

```swift
func additionner(a: Int, b: Int) -> Int {
    return a + b
}

func soustraire(a: Int, b: Int) -> Int {
    return a - b
}

func multiplier(a: Int, b: Int) -> Int {
    return a * b
}
```

Toutes ces fonctions ont le même type :

```swift
(Int, Int) -> Int
```

15. Changer la fonction utilisée

```swift
var operation: (Int, Int) -> Int
operation = additionner
print(operation(10, 5))
```

Résultat :

```
Console
15
```

Puis :

```swift
operation = multiplier
print(operation(10, 5))
```

Résultat :

```
Console
50
```

La variable peut donc représenter différentes fonctions compatibles.

16. Pourquoi est-ce utile ?

Dans une application mobile, le comportement à exécuter peut varier.

Contexte	Variation
`Bouton`	Action différente selon le contexte
`Calcul`	Stratégie différente
`Validation`	Règle différente
`Tri`	Critère différent
`Traitement réseau`	Action différente selon le résultat

Swift permet de transmettre un comportement comme une donnée.

17. Une fonction peut recevoir une autre fonction

Exemple :

```
func calculer(
    a: Int,
    b: Int,
    operation: (Int, Int) -> Int
) -> Int {
    return operation(a, b)
}
```

Cette fonction reçoit deux nombres et une fonction.

18. Utiliser une fonction comme paramètre

```
func additionner(a: Int, b: Int) -> Int {
    return a + b
}

func calculer(
    a: Int,
    b: Int,
    operation: (Int, Int) -> Int
) -> Int {
    return operation(a, b)
}
```

Appel :

```
let resultat = calculer(
    a: 10,
    b: 5,
    operation: additionner
)
print(resultat)
```

Résultat :

```
Console
15
```

19. Exemple avec plusieurs comportements

```
func additionner(a: Int, b: Int) -> Int {
    return a + b
}

func multiplier(a: Int, b: Int) -> Int {
    return a * b
}
```

Puis :

```
print(
    calculer(a: 10, b: 5, operation: additionner)
)
```

Résultat :

```
Console
15
```

Avec :

```
print(
    calculer(a: 10, b: 5, operation: multiplier)
)
```

Résultat :

```
Console
50
```

20. Exemple — Traitement d'un niveau de batterie

On peut créer plusieurs fonctions :

```
func afficherPourcentage(_ niveau: Int) {
    print("Batterie : \(niveau)%")
}

func afficherAlerte(_ niveau: Int) {
    if niveau <= 20 {
        print("Attention : batterie faible")
    }
}
```

Puis une fonction générale :

```
func traiterBatterie(
    niveau: Int,
    action: (Int) -> Void
) {
    action(niveau)
}
```

Utilisation :

```
traiterBatterie(
    niveau: 15,
    action: afficherPourcentage
)
```

Résultat :

```
Console
Batterie : 15%
```

Puis :

```
traiterBatterie(
    niveau: 15,
    action: afficherAlerte
)
```

Résultat :

```
Console
Attention : batterie faible
```

La même fonction générale peut donc exécuter différents traitements.

21. Une fonction peut retourner une fonction

Swift permet également à une fonction de retourner une autre fonction.

Exemple :

```
func augmenter(_ valeur: Int) -> Int {
    return valeur + 1
}

func diminuer(_ valeur: Int) -> Int {
    return valeur - 1
}

func choisirOperation(augmenterValeur: Bool) -> (Int) -> Int {
    if augmenterValeur {
        return augmenter
    } else {
        return diminuer
    }
}
```

Utilisation :

```
let operation = choisirOperation(
    augmenterValeur: true
)
let resultat = operation(10)
print(resultat)
```

Résultat :

```
Console
11
```

22. Lire un type de fonction

Il est important de savoir lire :

```
(Int) -> Int
```

```
1 paramètre Int
      ↓
1 résultat Int
```

```
(String, Int) -> Bool
```

```
String + Int
      ↓
    Bool
```

```
(Int) -> Void
```

```
Int
 ↓
aucune valeur retournée
```

23. Fonctions imbriquées

Swift permet de déclarer une fonction à l'intérieur d'une autre fonction.

Exemple :

```swift
func traiterConnexion(estConnecte: Bool) {
    func afficherSucces() {
        print("Connexion réussie")
    }
    func afficherErreur() {
        print("Connexion impossible")
    }
    if estConnecte {
        afficherSucces()
    } else {
        afficherErreur()
    }
}
```

24. Pourquoi utiliser une fonction imbriquée ?

Une fonction imbriquée peut être utile lorsque le traitement n'est nécessaire qu'à l'intérieur d'une seule autre fonction. Cela permet de limiter sa portée.

Dans :

```
func traiterConnexion(...) {
    func afficherErreur() {
    }
}
```

afficherErreur() n'est accessible que dans traiterConnexion().

25. Exemple — Validation d'un code

```
func validerAcces(code: Int) {
    func succes() {
        print("Accès autorisé")
    }
    func erreur() {
        print("Code incorrect")
    }
    if code == 1234 {
        succes()
    } else {
        erreur()
    }
}
```

Utilisation :

```
validerAcces(code: 1234)
```

26. Paramètres inout

Par défaut, les paramètres d'une fonction sont des constantes locales.

Considérons :

```
func incrementer(nombre: Int) {
    nombre += 1
}
```

Ce code provoque une erreur. Pourquoi ? Le paramètre nombre ne peut pas être modifié directement.

27. Modifier directement une variable externe

Swift propose :

```
inout
```

Exemple :

```
func incrementer(nombre: inout Int) {
    nombre += 1
}
```

Utilisation :

```
var compteur = 10
incrementer(nombre: &compteur)
print(compteur)
```

Résultat :

```
Console
11
```

28. Le symbole &

Avec un paramètre inout, l'appel utilise :

```
&
```

Exemple :

```
incrementer(nombre: &compteur)
```

Cela signifie que la fonction peut modifier directement la variable originale.

29. Exemple — Modifier un niveau sonore

```
func augmenterVolume(
    volume: inout Int,
    de valeur: Int
) {
    volume += valeur
    if volume > 100 {
        volume = 100
    }
}
```

Utilisation :

```
var volume = 80
augmenterVolume(
    volume: &volume,
    de: 30
)
print(volume)
```

Résultat :

```
Console
100
```

30. Quand utiliser inout ?

inout est utile lorsqu'une fonction doit réellement modifier la variable reçue. Cependant, il ne faut pas l'utiliser systématiquement.

Souvent, cette approche est plus simple :

```
func incrementer(nombre: Int) -> Int {
    return nombre + 1
}
```

Puis :

```
compteur = incrementer(nombre: compteur)
```

Les deux approches existent. Le choix dépend du contexte.

31. Exemple complet — Gestion d'une progression

```
func augmenterProgression(
    progression: inout Int,
    de valeur: Int
) {
    progression += valeur
    if progression > 100 {
        progression = 100
    }
}

func obtenirEtatProgression(
    progression: Int
) -> (
    terminee: Bool,
    message: String
) {
    if progression >= 100 {
```

```
        return (true, "Téléchargement terminé")
    }
    return (
        false,
        "Téléchargement en cours"
    )
}
```

Utilisation :

```
var progression = 70
augmenterProgression(
    progression: &progression,
    de: 20
)

let etat = obtenirEtatProgression(
    progression: progression
)

print("Progression : \(progression)%")
print(etat.message)
```

32. Exemple - Simulateur d'opérations

Créer plusieurs fonctions :

```
func additionner(_ a: Double, _ b: Double) -> Double {
    return a + b
}

func soustraire(_ a: Double, _ b: Double) -> Double {
    return a - b
}

func multiplier(_ a: Double, _ b: Double) -> Double {
    return a * b
}
```

Puis :

```
func executerOperation(
    a: Double,
    b: Double,
    operation: (Double, Double) -> Double
) -> Double {
    return operation(a, b)
}
```

Utilisation :

```
let resultat = executerOperation(
    a: 20,
    b: 5,
    operation: multiplier
)
print(resultat)
```

33. Exercice - Validation d'une connexion mobile

Créer une fonction :

```
func verifierConnexion(
    utilisateur: String,
    motDePasse: String
) -> (
    succes: Bool,
    message: String
)
```

Règles :

Condition	Résultat
`Utilisateur vide`	false — "Nom d'utilisateur requis"
`Mot de passe vide`	false — "Mot de passe requis"
`Utilisateur = "admin" ET mot de passe = "1234"`	true — "Connexion réussie"
`Sinon`	false — "Identifiants incorrects"

Afficher ensuite le résultat retourné par la fonction.

34. Exercice - Opérations sur le niveau de batterie

Créer les fonctions suivantes :

```
func augmenter(_ niveau: Int) -> Int
```

et :

```
func diminuer(_ niveau: Int) -> Int
```

Puis créer :

```
func modifierBatterie(
    niveau: Int,
    operation: (Int) -> Int
) -> Int
```

Tester la fonction avec les deux comportements.

Règles :

- Le niveau ne doit jamais dépasser 100
- Le niveau ne doit jamais être inférieur à 0

35. Exercice - Fonction de traitement

Créer deux fonctions :

```
func afficherConnexion(_ message: String)
```

et :

```
func afficherErreur(_ message: String)
```

Créer ensuite :

```
func traiterMessage(
    message: String,
    action: (String) -> Void
)
```

La fonction traiterMessage doit exécuter la fonction reçue dans action. Tester les deux traitements.

36. Exercice - Choisir une opération

Créer :

```
func choisirOperation(
    type: String
) -> (Double, Double) -> Double
```

Valeurs possibles :

```
"addition"
"soustraction"
"multiplication"
```

La fonction doit retourner la fonction appropriée.

Exemple :

```
let operation = choisirOperation(
    type: "multiplication"
)
let resultat = operation(5, 4)
print(resultat)
```

Résultat :

```
Console
20
```

37. À retenir

Une fonction Swift peut retourner plusieurs valeurs grâce à un tuple :

```
func fonction() -> (
    resultat: Bool,
    message: String
)
```

Une fonction possède elle-même un type :

```
(Int, Int) -> Int
```

Elle peut être stockée dans une variable :

```
let operation = additionner
```

Elle peut être transmise à une autre fonction :

```
func calculer(
    operation: (Int, Int) -> Int
)
```

Elle peut également être retournée par une fonction :

```
func choisirOperation() -> (Int) -> Int
```

Swift permet aussi les fonctions imbriquées :

```
func externe() {
    func interne() {
    }
}
```

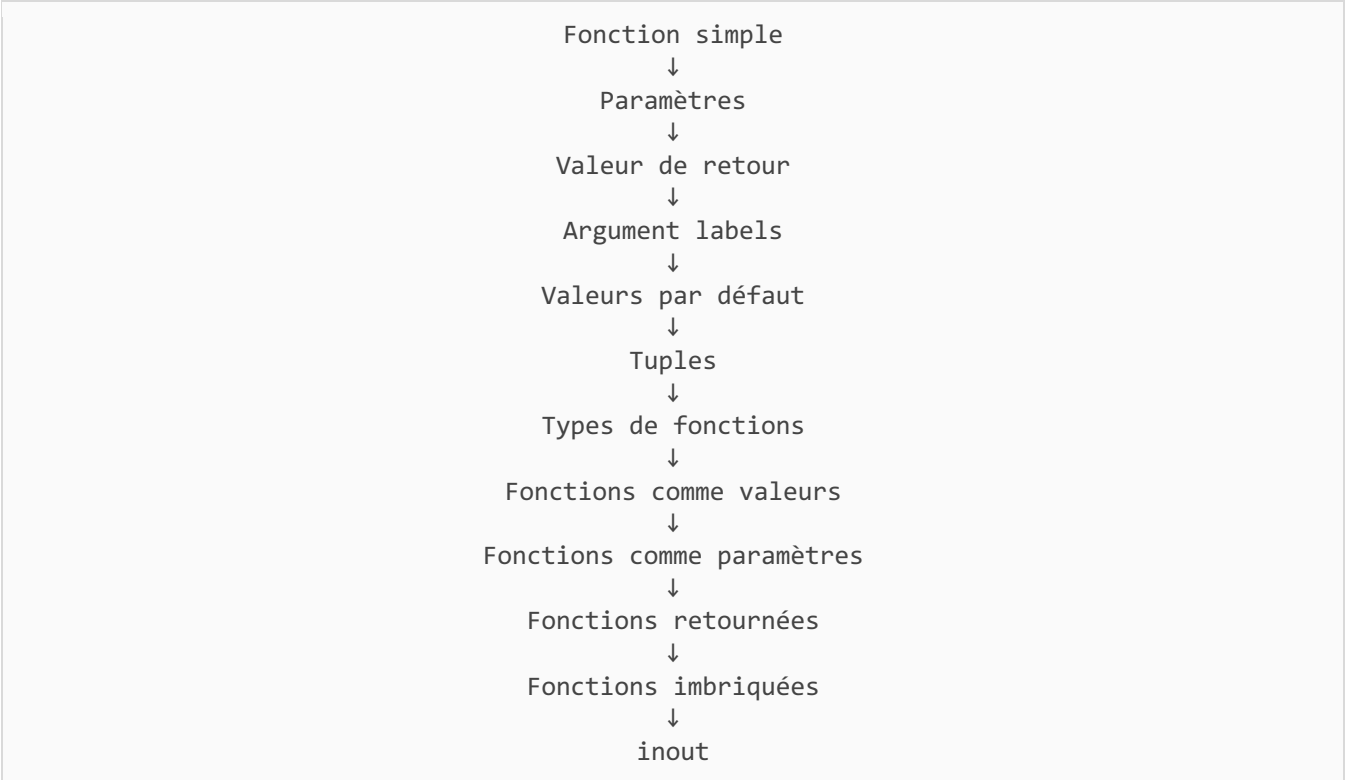
Et inout permet de modifier directement une variable reçue :

```
func modifier(valeur: inout Int)
```

avec :

```
modifier(valeur: &nombre)
```

38. Vision globale des fonctions en Swift



À ce stade : les fonctions ne sont plus seulement des blocs de code réutilisables - en Swift, elles peuvent également être manipulées comme des valeurs et transmises à d'autres parties du programme.